

SOFTWARE DESIGN SPECIFICATION

AI Comic Generator

Web-Based AI-Powered Comic Strip Generation System

Project Title	AI Comic Generator
Project Code	BSCS-FYP-2024
Internal Advisor	Professor Saad Razzaq
External Advisor	Professor Saad Razzaq
Project Team	Abdullah Akram (BSCS51F22S058) Laraib Rehman (BSCS51F22S077) Muhammad Ahmed (BSCS51F22S085)
Submission Date	December 2024
Document Version	1.0
Status	Final
University	University of Sargodha (UOS)

Project Manager's Signature

Document Information

Category	Information
University	University of Sargodha (UOS)
Project	AI Comic Generator
Document	Software Design Specification
Document Version	1.0
Identifier	BSCS-FYP-2025
Status	Final
Author(s)	Abdullah Akram · Laraib Rehman · Muhammad Ahmed
Approver(s)	Project Manager / Academic Advisor
Issue Date	December 2025
Document Location	Project Repository / Submission Folder
Distribution	Advisor PM Project Office Examiner

Definition of Terms, Acronyms and Abbreviations

This section provides definitions of all terms, acronyms, and abbreviations required to properly interpret this document.

Term / Acronym	Description
AI	Artificial Intelligence — simulation of human intelligence by computer systems
API	Application Programming Interface — a set of protocols for building software applications
ASGI	Asynchronous Server Gateway Interface — Python async web server standard
CORS	Cross-Origin Resource Sharing — HTTP mechanism allowing cross-domain requests
DFD	Data Flow Diagram — graphical representation of data flow through a system
ERD	Entity Relationship Diagram — database schema visualization
FastAPI	Modern Python web framework for building REST APIs with automatic documentation
FYP	Final Year Project
GUI	Graphical User Interface
HTML	HyperText Markup Language — standard language for web documents
JSX	JavaScript XML — syntax extension for React allowing HTML-like code in JavaScript
MVC	Model-View-Controller — software architectural pattern
ORM	Object-Relational Mapping — maps database tables to programming objects
OTP	One-Time Password — single-use verification code sent via email
PNG	Portable Network Graphics — lossless image format used for comic downloads
React	JavaScript library for building component-based user interfaces (Meta)
REST	Representational State Transfer — architectural style for distributed systems
SDS	Software Design Specification — this document
SMTP	Simple Mail Transfer Protocol — standard for email transmission
SQLite	Lightweight file-based relational database management system
UML	Unified Modeling Language — standard visual language for software modeling

Term / Acronym	Description
UOS	University of Sargodha

Table of Contents

Section
1. System Architecture
1.1 System Level Architecture
1.2 Sub-System / Component / Module Level Architecture
1.3 Sub-Component / Sub-Module Level Architecture
2. Design Strategies
2.1 Architectural Design Strategy
2.2 Data Management Strategy
2.3 AI Integration Strategy
2.4 Security & Authentication Strategy
2.5 UI/UX Design Strategy
3. Detailed System Design
3.1 Entity Relationship Diagram (ERD)
3.2 Use Case Diagram
3.3 State Transition Diagram
3.4 Sequence Diagram
3.5 Data Flow Diagram (DFD — Level 0 & Level 1)
3.6 Collaboration / Communication Diagram
3.7 Class Diagram
3.8 Activity Diagram

1. System Architecture

This section provides a high-level overview of how the functionality and responsibilities of the AI Comic Generator system are partitioned and assigned to subsystems and components. The primary objective is to establish a general understanding of how the system is decomposed and how individual parts cooperate to provide the desired functionality.

1.1 System Level Architecture

The AI Comic Generator follows a three-tier client-server architecture. The system is decomposed into three major tiers that interact through well-defined interfaces:

- **Presentation Tier (Frontend):** A React.js Single Page Application (SPA) running in the user's web browser. Responsible for all UI rendering, user interaction handling, client-side routing, local session management, and comic download functionality.
- **Application Tier (Backend):** A Python FastAPI server providing a RESTful API. Responsible for authentication logic, OTP generation and verification, story processing, AI image orchestration, and database operations.
- **Data Tier (Database):** A SQLite file-based relational database (comic_generator.db) persisting user account records and saved comic data.

Additionally, one external system is integrated:

- **Email Service — Gmail SMTP:** Used by the backend to dispatch OTP verification emails to newly registered users.

System-level component relationships:

Frontend (React.js)	↔	Backend (FastAPI)	↔	Database (SQLite)
Backend ↔ AI Image API		Backend	↔	Gmail SMTP (Email Service)

Interface boundaries:

- **Frontend ↔ Backend:** HTTP/REST over localhost (port 3000 → 5000); JSON payload; Axios client with 120-second timeout; CORS restricted to http://localhost:3000.
- **Backend ↔ Pollinations.ai:** HTTPS GET requests with URL-encoded prompts; direct image URL returned.
- **Backend ↔ Gmail SMTP:** SMTPS/STARTTLS with App Password authentication.

1.2 Sub-System / Component / Module Level Architecture

The system is further decomposed into the following modules:

1.2.1 Frontend Modules

Module / Component	Route	Responsibility
LoadingScreen	(pre-route)	Marvel-style animated intro on first load; skip button; phase management
LandingPage	/	Marketing homepage: hero, features, stats, testimonials, footer
Login	/login	Authentication form; credential submission; session storage on success
Signup	/signup	Registration form; email/password validation; OTP trigger
VerifyOTP	/verify-otp	6-digit OTP entry; auto-format; resend with countdown timer
ComicGenerator	/generator	Core workflow: input → generating → preview; story input; panel selection; download
MyComics	/my-comics	localStorage-based gallery; view/delete saved comics
About	/about	Project info; team member cards; technology stack
services/api.js	(utility)	Axios instance; all API call functions (loginUser, signupUser, generateComic, etc.)

1.2.2 Backend Modules

Module	Responsibility
main.py	FastAPI app instantiation; CORS middleware; all route/endpoint definitions; uvicorn server entry point
database.py	SQLAlchemy engine and session configuration; User and Comic ORM model definitions; CRUD helper functions (create_user, get_user_by_email, save_comic, etc.)
email_service.py	Gmail SMTP connection; 6-digit OTP generation; HTML email template construction; temporary in-memory OTP storage dictionary
.env	Environment-specific secrets: Gmail address, App Password; excluded from version control

1.2.3 External Service Modules

External Service	Integration Details
Gmail SMTP	SMTPS on port 465 App Password auth Sends OTP in HTML-formatted email Library: Python smtplib + email.mime

1.3 Sub-Component / Sub-Module Level Architecture

The ComicGenerator module — the most complex frontend component — is internally decomposed into three workflow states managed via React useState:

- State: 'input' — Renders the story textarea (with character counter) and panel count selector (2 / 4 / 6 / 8). The Generate button is disabled if the story field is empty. On submission, transitions to 'generating'.
- State: 'generating' — Displays an animated loading overlay. Calls services/api.js → POST /api/generate-comic. Awaits response; on success transitions to 'preview'; on failure shows error message and returns to 'input'.
- State: 'preview' — Renders the panel image grid using CSS Grid layout. Provides Download (html2canvas capture → PNG Blob) and Save to Gallery (localStorage write) actions. Provides a 'Generate Another' button to return to 'input'.

The backend main.py generate-comic endpoint is internally decomposed into two functions:

- split_story_into_scenes(story, panel_count): Splits input text on sentence-ending punctuation (. ! ?) using Python regex; groups sentences into N equal scene strings.
 - generate_panel_image(scene, panel_number): URL-encodes the scene string; prefixes with 'comic book panel style: '; constructs and returns the Pollinations.ai URL.
-

2. Design Strategies

This section describes the key design strategies and architectural decisions that shape the overall organization and high-level structure of the AI Comic Generator. Each strategy is accompanied by the rationale employed and relevant trade-offs considered during development.

2.1 Strategy 1 — Three-Tier Separation of Concerns

Decision: Strict separation between the presentation layer (React frontend), business logic layer (FastAPI backend), and data layer (SQLite database).

Rationale:

- Enables independent development: frontend and backend can be built and tested separately.
- Facilitates future migration — e.g., replacing SQLite with PostgreSQL or replacing the React frontend with a mobile app — without impacting other tiers.
- Aligns with industry-standard MVC/REST patterns, making the codebase familiar and maintainable.

Trade-offs:

- Requires running two separate server processes (`npm start` + `python main.py`) during development.
- Introduces CORS configuration overhead to allow cross-origin requests between port 3000 and 5000.

Future Extension: In a production deployment, the React build can be served as static files from the same server, eliminating CORS issues and simplifying the operational setup.

2.2 Strategy 2 — Free-Tier AI Image Generation

Decision: Use Pollinations.ai as the AI image generation service instead of paid alternatives (OpenAI DALL-E, Stability AI).

Rationale:

- Zero cost: Pollinations.ai is a free public API with no authentication requirement, making the project completely cost-free.
- No rate-limit complexity: No API key management, billing, or quota monitoring required.
- Sufficient quality for academic demonstration purposes.

Trade-offs:

- No Service Level Agreement (SLA): Image quality and availability depend on Pollinations.ai infrastructure.
- Image generation latency is variable — panels may take 5–15 seconds each depending on server load.
- Limited style control compared to paid APIs.

Future Extension: Integration with OpenAI DALL-E or Stable Diffusion API for production-grade image quality and consistent art styles (manga, western comic, etc.).

2.3 Strategy 3 — Stateless Backend with Client-Side Session

Decision: Use localStorage for session persistence instead of server-side sessions or JWT tokens.

Rationale:

- Simplifies backend architecture — no session store or token validation middleware required.
- Sufficient for a local academic demonstration where security is not a primary concern.
- Reduces backend complexity for a 2-3 week development timeline.

Trade-offs:

- localStorage is vulnerable to XSS attacks in a production setting.
- No true server-side session invalidation — logout only clears client-side data.

Future Extension: Implement JWT-based stateless authentication where the server signs tokens and the client includes them in request headers, enabling secure, scalable session management.

2.4 Strategy 4 — File-Based Database (SQLite)

Decision: Use SQLite as the database engine managed via SQLAlchemy ORM.

Rationale:

- File-based: No separate database server to install or configure. The entire database is a single .db file.
- Zero setup for evaluators: The database is initialized by running python database.py once.
- SQLAlchemy ORM provides clean Pythonic models and separates SQL from business logic.

Trade-offs:

- Not suitable for concurrent multi-user write workloads at scale.
 - No built-in network access — cannot be queried from a remote machine without file sharing.
-

Future Extension: Migrate to PostgreSQL for production; the SQLAlchemy ORM abstraction means the model code requires minimal changes — only the connection string in .env needs to be updated.

2.5 Strategy 5 — Component-Based Frontend with Utility-First CSS

Decision: React.js with Tailwind CSS v3 for all UI development.

Rationale:

- React's component model enforces single-responsibility UI units, making code reusable and testable.
- Tailwind CSS eliminates the need for custom CSS files — styles are applied inline via utility classes, dramatically speeding up development.
- React Router provides client-side navigation without full page reloads, delivering a smooth SPA experience.
- React Hooks (useState, useEffect, useNavigate, useLocation) provide clean state and lifecycle management without class components.

Trade-offs:

- Tailwind class lists in JSX can become verbose for complex components.
- Initial Tailwind configuration (tailwind.config.js, postcss.config.js) requires careful setup.

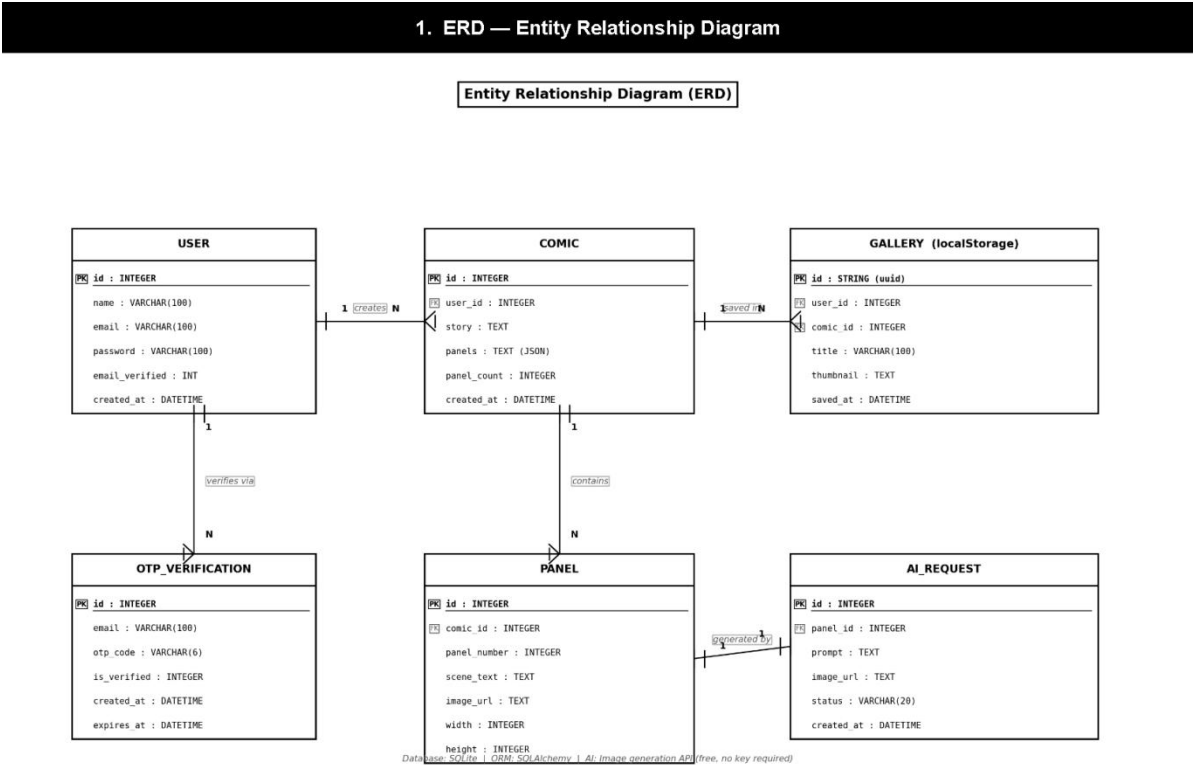
User Interface Paradigm: Glassmorphism — dark backgrounds with backdrop-blur, translucent panels, gradient purple-to-pink accents — providing a modern, professional visual identity consistent across all pages.

3. Detailed System Design

This section presents the complete set of UML and system design diagrams for the AI Comic Generator. Each diagram is accompanied by a detailed description of the elements, relationships, and design decisions it represents.

3.1 Entity Relationship Diagram (ERD)

Logical data model showing the database entities, their attributes, and relationships.



Description

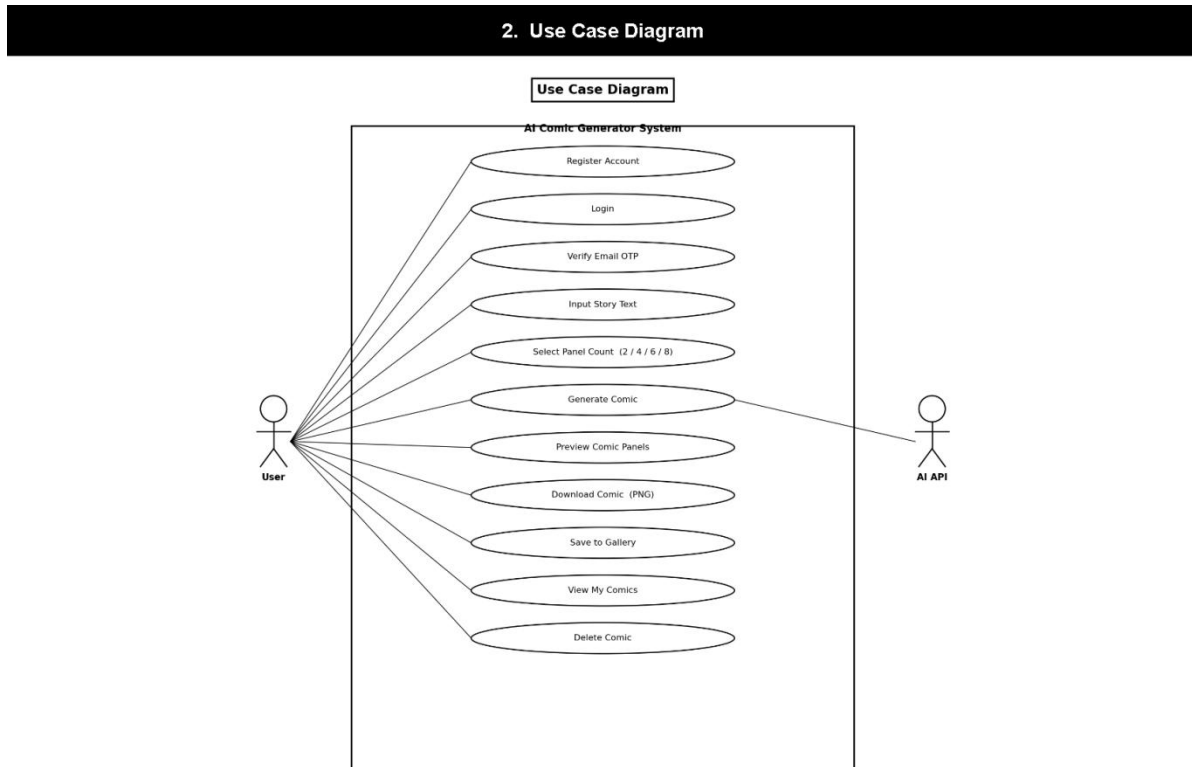
The database schema consists of two entities:

- User:** Represents a registered account. Attributes: id (PK), name, email (UNIQUE), password, email_verified (0/1), created_at.
- Comic:** Represents a generated comic strip saved by a user. Attributes: id (PK), user_id (FK → User.id), story (TEXT), panels (TEXT — JSON-serialized array of {id, imageUrl} objects), panel_count (INTEGER), created_at.

Relationship: One User can own zero or many Comics (ONE-TO-MANY). The user_id foreign key on the Comic table enforces referential integrity.

3.2 Use Case Diagram

Functional view showing system actors, use cases, and their relationships.



Description

Actors:

- Registered User (primary actor): A person with a verified account. Has full access to all system features.
- Guest User: An unauthenticated visitor. Can access Landing Page and About Page only.
- Email Service (external): Gmail SMTP; receives OTP dispatch requests from the system.
- AI Image Service (external): Pollinations.ai; receives image generation requests from the backend.

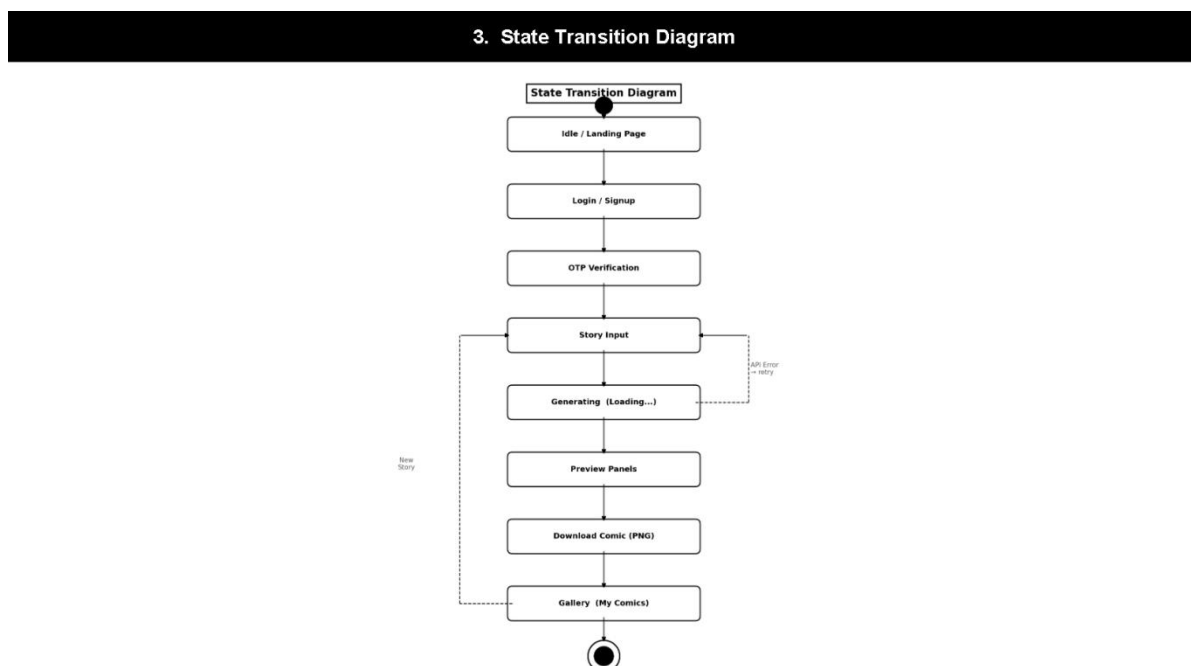
Core Use Cases:

- Register Account — Guest fills signup form; system validates and dispatches OTP.
- Verify Email (OTP) — Guest enters 6-digit code; system creates account on success.
- Login — Registered User authenticates; session stored in localStorage.

- Generate Comic — User submits story + panel count; system splits story, calls AI API, returns panel images.
- Download Comic — User captures comic grid as PNG via html2canvas.
- Save to Gallery — User saves comic data to localStorage.
- View Gallery (My Comics) — User browses, loads, or deletes saved comics.
- Logout — User clears session data from localStorage.

3.3 State Transition Diagram

Dynamic view showing the states of the ComicGenerator workflow and authentication flow, and the transitions between them.



Description

Authentication State Machine:

- GUEST → [Register] → OTP_PENDING → [Verify OTP] → REGISTERED → [Login] → AUTHENTICATED
- AUTHENTICATED → [Logout] → GUEST

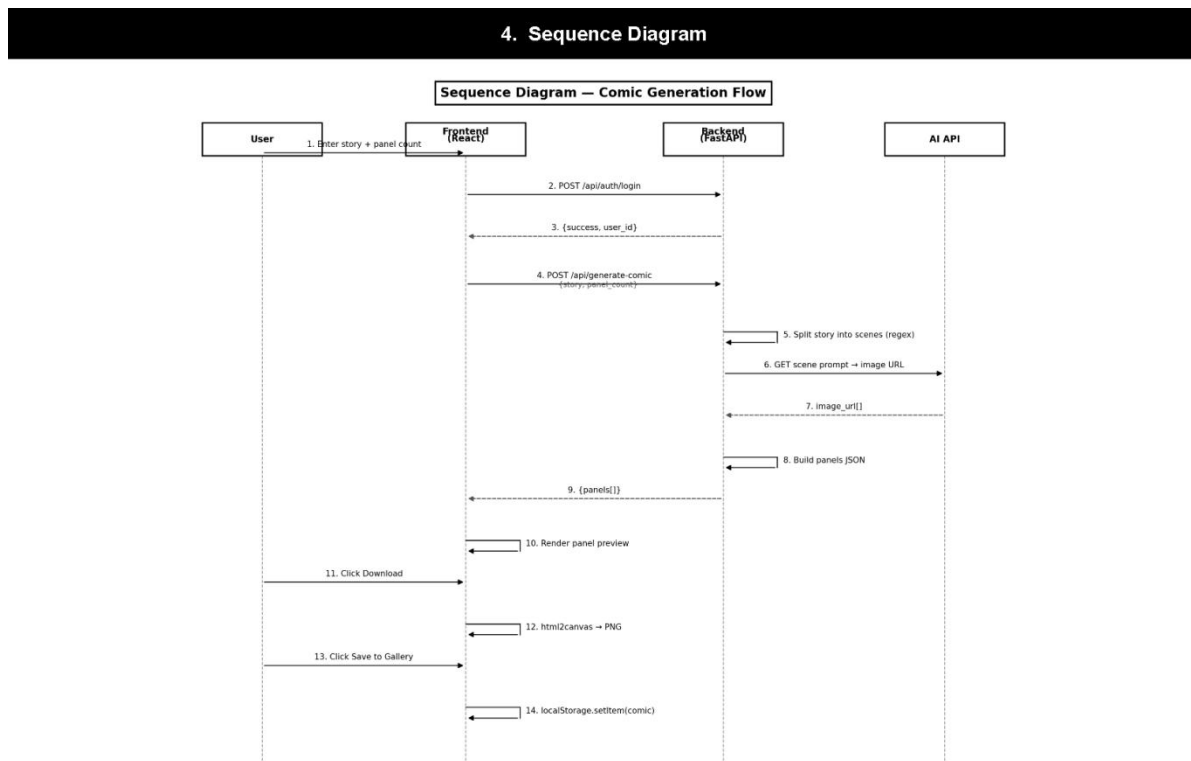
Comic Generation State Machine (ComicGenerator component):

- INPUT: Initial state. User enters story text and selects panel count. Transition: [Click Generate] → GENERATING

- GENERATING: Loading animation shown. API call in progress. Transitions: [Success] → PREVIEW | [Failure/Timeout] → INPUT (error shown)
- PREVIEW: Panel images displayed in grid. Transitions: [Download] → PREVIEW (no state change, file downloaded) | [Save] → PREVIEW (localStorage updated) | [Generate Another] → INPUT

3.4 Sequence Diagram

Interaction view showing the time-ordered message exchange between system objects during the comic generation workflow.



Description

The primary sequence — Comic Generation — involves the following participants:

- User (browser)
- ComicGenerator Component (React)
- services/api.js (Axios)
- FastAPI Backend (main.py)
- database.py
- Pollinations.ai (external)

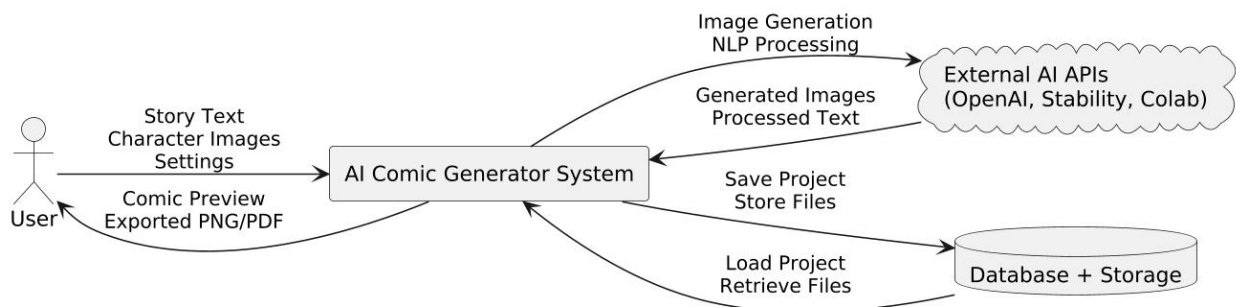
Message sequence:

#	From	To	Message
1	User	ComicGenerator	Submit story + panel_count
2	ComicGenerator	api.js	POST /api/generate-comic {story, panels}
3	api.js	FastAPI	HTTP POST request
4	FastAPI	database.py	Validate user session (user_id lookup)
5	FastAPI	FastAPI	split_story_into_scenes(story, N)
6..N+5	FastAPI	Pollinations.ai	GET image URL for each scene prompt
N+6	Pollinations.ai	FastAPI	Return image URL per panel
N+7	FastAPI	api.js	Return [{id, imageUrl}] array
N+8	api.js	ComicGenerator	Resolve promise with panels data
N+9	ComicGenerator	User	Render panel grid in PREVIEW state

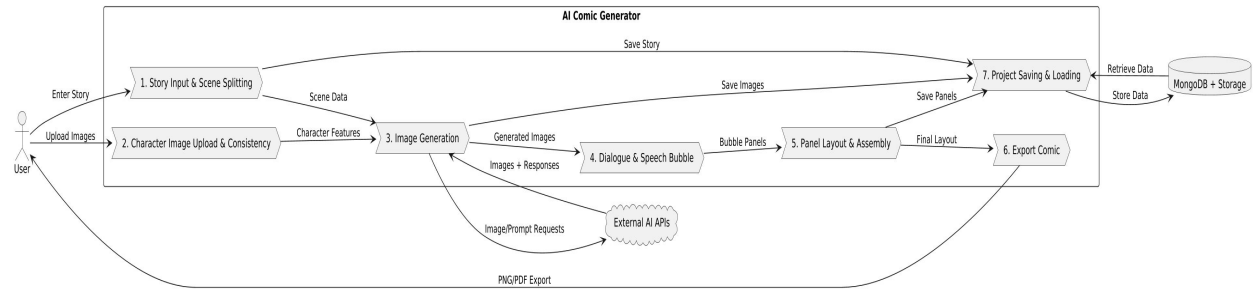
3.5 Data Flow Diagram (DFD — Level 0, Level 1, Level 2 & Level 3)

Process-oriented view showing how data moves through the system, from external entities through processes to data stores.

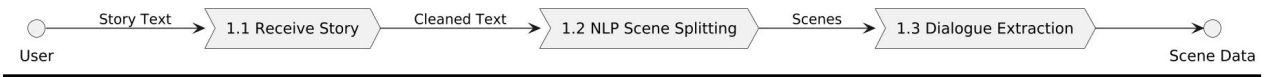
LEVEL 0 DFD Context Diagram



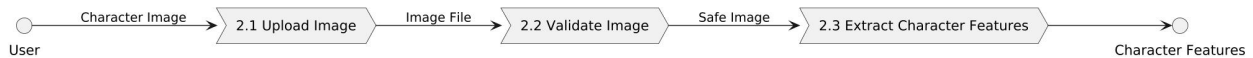
LEVEL 1 Main process DFD Diagram



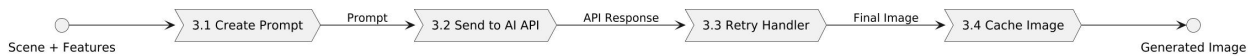
LEVEL-2 for Process 1 Story Input & Scene Splitting



LEVEL-2 for Process 2: Character Image Upload & Consistency



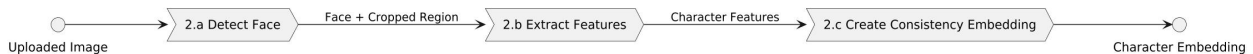
LEVEL-2 for Process 3 Image Generation



LEVEL-3 for (1) NLP Scene Splitting



LEVEL-3 for (2) Character Consistency



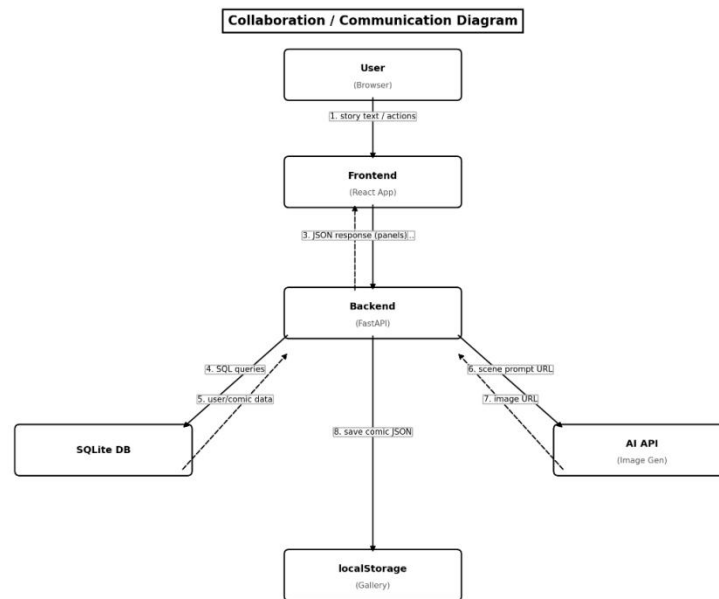
LEVEL-3 for (3) Image Generation Engine



3.6 Collaboration / Communication Diagram

Structural interaction view showing the objects involved in the system and the messages exchanged between them, emphasizing object relationships.

6. Collaboration / Communication Diagram



Description

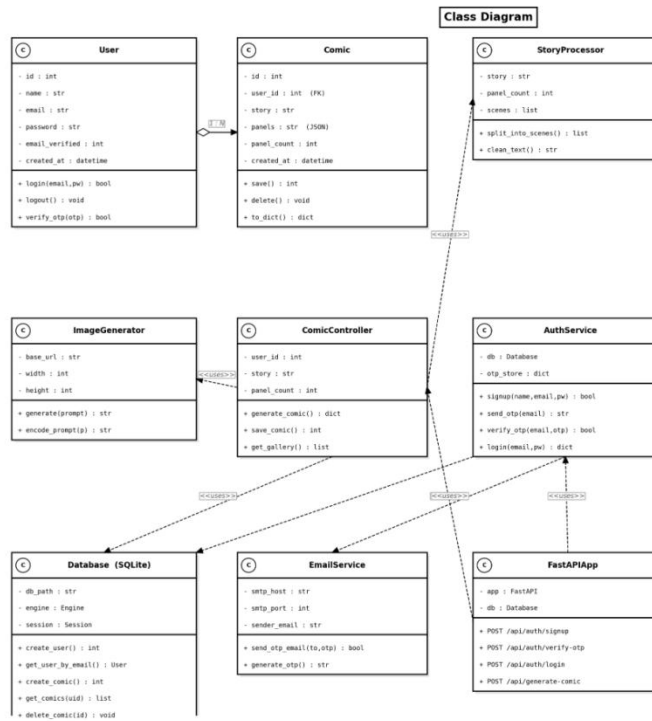
Objects and communication links:

- User Browser ↔ React Router: Navigation messages (route changes, redirects).
- React Components ↔ localStorage: Read/write for session data (user_id, name, email) and comics gallery.
- React Components ↔ services/api.js: Function calls for each API operation (loginUser, signUpUser, generateComic).
- services/api.js ↔ FastAPI Server: HTTP request/response messages over localhost:5000.
- FastAPI Server ↔ SQLAlchemy ORM: Python object calls translated to SQL queries on comic_generator.db.
- FastAPI Server ↔ email_service.py: Function calls for OTP generation and email dispatch.
- email_service.py ↔ Gmail SMTP: SMTP protocol messages for email delivery.
- FastAPI Server ↔ Pollinations.ai: HTTP GET requests for image generation.

3.7 Class Diagram

Static structural view showing the system's classes, their attributes, methods, and inter-class relationships.

7. Class Diagram



Description

Backend Classes (SQLAlchemy ORM Models):

Class	Attributes	Methods
User	id: Integer (PK) name: String(100) email: String(100) UNIQUE password: String(100) email_verified: Integer created_at: DateTime	create_user(name, email, password) get_user_by_email(email) verify_credentials(email, password)
Comic	id: Integer (PK) user_id: Integer (FK) story: Text panels: Text (JSON) panel_count: Integer created_at: DateTime	save_comic(user_id, story, panels, count) get_comics_by_user(user_id) delete_comic(comic_id)
OTPManager	otp_store: Dict {email: otp} expiry_store: Dict {email: timestamp}	generate_otp(): String store_otp(email, otp) verify_otp(email, otp): Boolean clear_otp(email)
ComicService	(stateless)	split_story_into_scenes(story, count): List[str] generate_panel_image(scene, n): String (URL) build_panels(story, count):

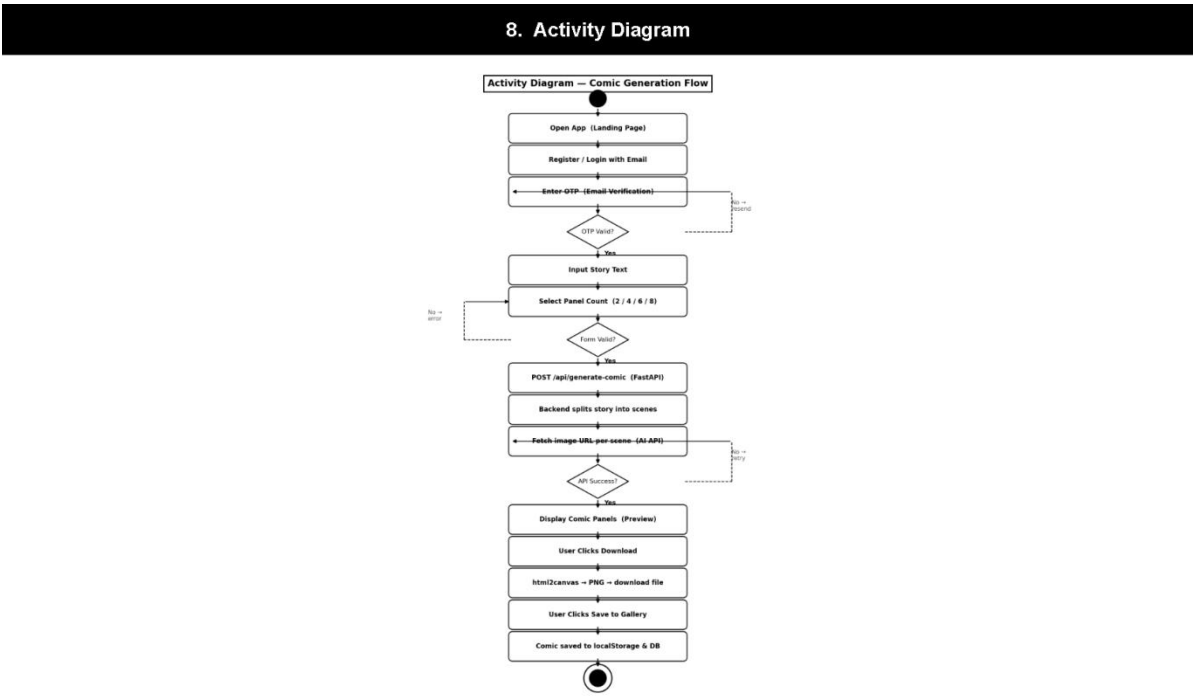
Class	Attributes	Methods
		List[{id, imageUrl}]
EmailService	smtp_host: String smtp_port: Integer sender_email: String	send_otp_email(recipient, otp) build_html_template(otp): String

Relationships:

- User 1 — * Comic (one-to-many association; user_id FK).
- ComicService uses EmailService indirectly through the FastAPI route handlers.
- OTPManager is used by the signup and verify-OTP route handlers in main.py.

3.8 Activity Diagram

Behavioral view showing the workflow of the complete user journey — from landing page through registration, login, comic generation, and gallery management.



Description

The activity diagram traces the following primary workflow:

Step	Activity
------	----------

Step	Activity
1	User visits application → Loading Screen animation displayed → Landing Page rendered
2	[New User] → Navigate to /signup → Fill registration form → Validate inputs → Send OTP email
3	Navigate to /verify-otp → Enter OTP → [Valid] → Account created → Navigate to /login [Invalid] → Retry / Resend
4	[Existing User] → Navigate to /login → Enter credentials → [Valid] → Store session in localStorage → Navigate to /generator [Invalid] → Show error
5	On /generator → Enter story text → Select panel count (2/4/6/8) → Click Generate
6	Backend: Split story → Generate image URLs (1 per panel via Pollinations.ai) → Return panel array
7	Frontend: Display panel image grid in PREVIEW state → [Download] → PNG saved locally [Save] → Written to localStorage
8	[View My Comics] → Navigate to /my-comics → Load gallery from localStorage → [View] → Load comic [Delete] → Remove from localStorage
9	[Logout] → Clear localStorage session → Navigate to /login

— End of Software Design Specification —

AI Comic Generator | Abdullah Akram · Laraib Rehman · Muhammad Ahmed | FYP 2024 | UOS
